

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 10. List

Lecture: 101. List Introduction

Section 1: Lecture Summary

This lecture provides a thorough introduction to Python lists, presenting them as a built-in, versatile data structure fundamental to Python programming. The instructor begins by highlighting the significance of lists in Python compared to arrays in other languages and outlines the topics covered: definition, creation methods, memory representation, heterogeneity, mutability, and key properties. The lecture demonstrates how lists can store various data types (heterogeneous), how to create lists using square brackets or the `list()` function, and explains the concept of list indexing including negative indices. The instructor further details how lists are stored in memory as arrays of references allowing heterogeneous elements. Then, mutability is explored by showing how to modify, add, and remove elements in lists. Finally, the lecture consolidates this knowledge into a precise definition of a list as an ordered, mutable collection of heterogeneous elements that can have duplicates.

Section 2: Key Concepts and Explanations

- What is a List?

- A list is a collection of elements grouped together.
- It is an ordered collection: elements have a fixed position/index.
- It can contain heterogeneous elements (different data types in the same list).
- It is mutable, meaning elements can be changed, added, or removed.
- Lists can have duplicate elements.

- Creating Lists in Python

- Using square brackets: e.g., `L1 = [1, 2, 3]`.
- Using the `list()` constructor with an iterable (e.g., tuple or string):
 - Example: `L2 = list((3, 5, 7, 9))` converts a tuple to a list.
 - Example: `L3 = list('abcde')` creates a list of characters.

- Creating an empty list: `L4 = []`.

- Heterogeneous Nature

- Lists can hold elements of multiple types including integers, floats, strings, booleans, complex numbers.

- Example: `L1 = [7, 3.2, "John", True, 5+6j]`.

- Internally, lists store references to objects; references are uniform so lists can hold any object type.

- Representation in Memory

- Lists are implemented as arrays of references pointing to the actual objects.

- Elements are stored contiguously, but each element points to a separate memory location.

- Indexing starts at 0 (positive indices) and can also be accessed using negative indices starting at -1 (last element).

- Mutability

- You can change elements at a specific index.

- You can add elements using methods like `.append()`.

- You can remove elements using methods like `.remove()`.

- Example: Changing an element value by assignment `L1[2] = 15`.

- Traversal

- Lists can be traversed using loops such as a `for` loop.

- Example: `for x in L1: print(x)` prints each element in the list.

- Indexing and Access

- Access elements by zero-based indices, e.g., `L1`.

- Negative indices allow access starting from the end, e.g., `L1[-1]` is the last element.

Section 3: Example Code and Use Cases

Creating lists using square brackets

```
L1 = [1, 2, 3, 4, 5, 6]  
print(L1) # Output: [1, 2, 3, 4, 5, 6]
```

Basic creation of a list of integers using square brackets.

Creating a list from a tuple using list()

```
L2 = list((3, 5, 7, 9))  
print(L2) # Output: [3, 5, 7, 9]
```

List created by converting a tuple to a list.

Creating a list from a string using list()

```
L3 = list('abcde')  
print(L3) # Output: ['a', 'b', 'c', 'd', 'e']
```

Converts string into a list of characters.

Heterogeneous list example

Output: [7, 3.2, 'John', True, (5+6j)]

```
L1 = [7, 3.2, 'John', True, 5+6j]  
print(L1)
```

Demonstrates list holding mixed data types.

Accessing elements using positive and negative indices

```
print(L1[2]) # Output: 'John'  
print(L1[-1]) # Output: (5+6j)
```

Accessing elements by positive and negative indices.

Traversing list elements using a for loop

```
for x in L1:  
    print(x)
```

Prints each element in the list.

Modifying an element (mutation)

```
L1 = [2, 4, 6, 8, 10, 12]  
L1[2] = 15  
print(L1) # Output: [2, 4, 15, 8, 10, 12]
```

Changes value at index 2.

Adding an element using append()

```
L1.append(25)
print(L1) # Output: [2, 4, 15, 8, 10, 12, 25]
```

Appends 25 to the end of the list.

Removing an element using remove()

```
L1.remove(10)
print(L1) # Output: [2, 4, 15, 8, 12, 25]
```

Removes the element with value 10.

Section 4: Key Takeaways

- A Python list is an **ordered collection** of elements that supports indexing.
- Lists are **heterogeneous**: able to hold different data types simultaneously.
- Lists are **mutable**, enabling changing, adding, and removing elements.
- Lists can contain **duplicates**.
- Lists can be created using **square brackets** or the `list()` function with an iterable.
- Lists are stored as an **array of references** internally, enabling heterogeneity.
- List elements can be accessed by **positive or negative indices**.
- Python provides built-in list **methods** such as `append()`, `remove()`, and others for list manipulation.
- Loops (`for`) are commonly used to **traverse** all elements in the list.

These fundamentals enable powerful, flexible use of lists in Python programming and form a foundation for understanding other data structures like tuples, sets, and dictionaries.